Prompt-Based Collaborative Agent Framework (PCAF) for Enhanced LLM Mathematical Reasoning

Petros Tsitsekidis
Department of Computer Science
Aarhus University
au802942@uni.au.dk

Abstract

The inherent unreliability of Large Language Models (LLMs) in complex, multi step calculations necessitates architectural interventions to achieve trustworthy accuracy in mathematical reasoning. This paper introduces the Prompt Based Collaborative Agent Framework (PCAF), a novel, architecture driven approach aimed at significantly improving final answer accuracy on the MathArena benchmark suite, including AIME 2025, HMMT (Feb/Nov) 2025, BRUMO 2025, SMT 2025, and CMIMC 2025. The method utilizes a single LLM instance (DeepSeek-V3-03-24) and employs sequential, role specific prompting to simulate a sophisticated, three agent collaborative system (Solver, Verifier, and Planner). This architecture transforms the LLM's linear reasoning into a persistent self correction mechanism. Through systematic evaluation across six distinct competition formats, we establish that the PCAF's structured intervention yields a significant uplift in global accuracy compared to the MathArena Zero Shot CoT baseline (53.65% versus 39.5%), with a 38.9% recovery rate confirming the efficacy of deterministic, architecture driven reflection.

1 Introduction

The pursuit of reliable mathematical reasoning in Large Language Models is hindered by two persistent challenges: computational imprecision and stubbornness in self correction. While LLMs excel at generating coherent solution narratives (Chain of Thought), they frequently fail at the final, precise steps during arithmetic, algebra, and complex counting, which leads to high rates of final answer error on benchmarks like MathArena. Furthermore, models typically struggle to effectively diagnose and pivot from their initial logical errors, making standard self correction techniques unreliable.

To overcome these limitations, this project implements the Prompt Based Collaborative Agent Framework (PCAF). Instead of relying on model fine tuning, PCAF utilizes an architecture driven reasoning approach. It simulates a multi agent system within a single LLM instance through sequential, role specific prompting. The framework dynamically cycles the LLM through three specialized roles. The Solver, the Verifier and the Planner which work in an iterative, closed loop refinement process. The core objective is to demonstrate that this structured collaborative architecture significantly increases the final answer accuracy and trustworthiness compared to the model's standard zero shot baseline provided by MathArena [**?**].

The key contributions of this work are:

- **Novel Algorithmic Structure:** We introduce the PCAF loop, which formalizes the iterative roles of Generation (Solver), Critique (Verifier), and Coordination (Planner), forcing systematic logical scrutiny.

Preprint.

- **Enforced Grounding:** We implement the mandatory integration of a persistent Python sandbox environment ("Code" tool) to enforce dual method verification (analytical vs. naive brute force) ensuring mathematical conclusions are derived from verifiable, executed code.
- **Targeted Improvement:** We apply and evaluate PCAF on a broad spectrum of challenging final answer subsections of the MathArena benchmark to provide a clear, quantitative measure of accuracy uplift across varied mathematical domains.

2 Related Work

The Prompt-Based Collaborative Agent Framework (PCAF) is informed by two established and growing areas of research in LLMs: Reflective Reasoning and Verification, and Multi-Agent Systems.

2.1 Reflective Reasoning and Verification

Early methods in mathematical reasoning focused on generating a single, detailed Chain-of-Thought (CoT). However, recent work emphasizes enhancing reliability through self-critique. Approaches such as **MA-LoT** by Wang et al. (2025) [3] and **KG-Prover** by Li et al. (2025) [1], which augments agents with knowledge graphs, highlight the value of structured introspection. These systems often succeed in generating multiple solution paths or utilizing formal methods to check internal consistency. However, a common limitation is *reasoning inertia*; when a model critiques itself without external grounding, it often doubles down on its initial hallucination. PCAF addresses this by introducing the `PersistentSolverSandbox`, enforcing that all self-corrections are grounded in executed code rather than purely linguistic introspection.

2.2 Multi-Agent Systems and Formal Verification

A parallel line of inquiry involves distributing tasks among distinct, specialized LLM instances to simulate collaborative environments. Works like Varambally et al. (2025) [2] employ specialized agents (e.g., a "formal prover" and a "critic") to ensure soundness. While computationally powerful, this multi-model approach requires complex orchestration and high inference costs. PCAF bridges this gap by simulating a multi-agent environment within a *single* model context. By using role-specific prompting to switch between Solver, Verifier, and Planner personas, PCAF achieves the diversity benefits of multi-agent systems while maintaining the deployment simplicity of a single inference stream.

3 Methodology

3.1 Framework Overview

The PCAF operates as a deterministic, closed-loop system designed to enforce self-correction. The architecture operates with a fixed budget of $T = 4$ total turns (one initial generation followed by up to three correction cycles).

The core cycle involves three sequential steps:

1. **Attempt (Solver):** The Solver generates a solution and a corresponding Python execution trace.
2. **Verification (Verifier):** The Verifier audits the trace against a structured checklist.
3. **Correction (Planner):** If the Verifier rejects the solution, the Planner translates the critique into a mandatory, actionable instruction for the next attempt.

The system employs an **Early Stopping** mechanism: the loop terminates immediately if the Verifier outputs a valid signal, minimizing token usage.

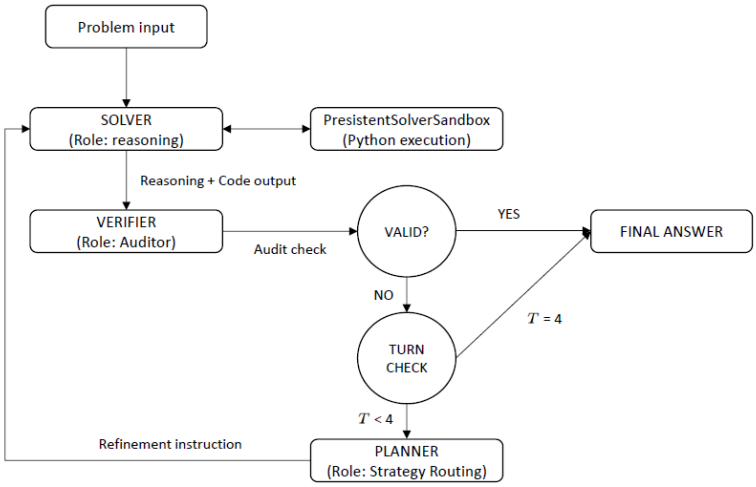

Figure 1: PCAF System Architecture

3.2 The Agents

3.2.1 The Solver

The Solver is the generative component, constrained by a strict system prompt `get_solver_prompt` that enforces the Dual-Verification Protocol for grounding and reliability.

"Two Strictly Different Methods" Mandate: The Solver must use two conceptually distinct approaches to arrive at the same answer:

- **Analytical Method (Method 1):** Utilizes formal mathematics, algebra, or efficient libraries (e.g., `sympy`).
- **Naive Brute-Force (Method 2):** Requires a simple, unoptimized Python script to simulate or iterate through all possibilities.

"Code-First" Mandate: The Solver is required to execute the code for both methods and explicitly compare the outputs, ensuring the final answer is a value confirmed by an external tool. This process is crucial for preventing numerical hallucinations.

3.2.2 The Verifier

The Verifier's role is that of a Skeptical Math Auditor (`VERIFIER_ROLE`). It receives the Solver's full reasoning and code outputs and audits the solution based on a strict checklist. To ensure the output is machine-parsable and deterministic, the Verifier is constrained via the LLM API's **JSON mode** (`response_format={"type": "json_object"}`) to output a strict schema:

```
{
  "valid": boolean,
  "error_type": "LOGIC_FLAW" | "VALUE_MISMATCH" | "METHOD_CONFLICT",
  "critique": string
}
```

The `error_type` field serves as the primary signal for the Planner, classifying failures into three key categories:

`LOGIC_FLAW` A fundamental error in the mathematical approach or application of a principle (e.g., misapplying the Principle of Inclusion-Exclusion or failing to handle edge cases like division by zero).

`METHOD_CONFLICT` A direct contradiction between the two mandated methods. This flag is triggered if the Analytical derivation yields one result (e.g., 42) while the Naive Brute-Force simulation yields another (e.g., 40).

`VALUE_MISMATCH` The Python code executed correctly, but the Solver's final textual conclusion ignored the code's output. This detects instances where the model "hallucinates" a preferred number despite contradictory empirical evidence from the sandbox.

3.2.3 The Planner

The Planner is a deterministic function (`construct_planner_feedback`) that routes error signals into rigid instructional templates. Its primary goal is to break the "reasoning inertia" often observed in LLMs. The logic implements three routing strategies:

`Syntax Repair (for Runtime Errors)` If the execution trace contains runtime errors (captured via the sandbox's standard error buffer, e.g., `IndentationError` or `Timeout`) or fails to produce output, the Planner ignores the mathematical content entirely. Instead, it issues a strict syntax directive: "Fix the syntax. Make the code run."

`Protocol Reset (for VALUE_MISMATCH)` To counteract the model's tendency to cling to a hallucinated number despite contradictory code output, this flag triggers a "Memory Wipe." The Planner instructs the Solver to: "Ignore your previous text intuition. It was wrong. Re-execute code and accept the output as truth."

`General Critique Relay (for LOGIC_FLAW / METHOD_CONFLICT)` For fundamental reasoning errors or conflicts between the two methods, the Planner avoids enforcing rigid strategy shifts (e.g., forcing Coordinate Geometry), as experimentation showed this often led to the model attempting to "game" the system (hardcoding). Instead, it relays the Verifier's specific critique and mandates a resolution of the inconsistency between the text and the code.

This architecture ensures the correction loop is not a generic "try again", but a precise, planned intervention.

3.3 Tool Use

The execution of mathematical code is handled by a custom, stateful environment, the `PersistentSolverSandbox`. This tool is essential for providing verifiable grounding:

Persistent Environment: Unlike typical stateless code execution tools, the sandbox maintains its global state (`self.globals`) across consecutive code blocks within a single Solver attempt. This enables complex, multi-block workflows in which a function is defined in one block and then invoked or modified in subsequent blocks.

Comprehensive Imports: The sandbox is pre-loaded with essential mathematical and symbolic libraries (`math`, `sympy`, `numpy`) to support both analytical and naive solution methods.

Robust I/O Capture: The sandbox reliably captures all standard output (`stdout`) and handles common runtime issues (e.g., infinite loops via strict 20-second timeouts, `IndentationError`, and `SyntaxError`), returning any error messages to the Solver as an `OBSERVATION`. This feedback loop is critical, as it allows the Solver to debug its own code syntax and execution failures.

4 Experiments

The central goal of these experiments is to quantitatively assess the performance gain of the PCAF architecture relative to the standard MathArena baseline. By applying PCAF to subsets where the baseline model either has established scores (AIME 2025, HMMT Feb 2025) or there is no score available (BRUMO 2025, SMT 2025, CMIMC 2025, HMMT Nov 2025), we establish a rigorous comparative framework to measure the impact of multiagent collaboration on final answer accuracy.

4.1 Experimental Setup

Implementation. The framework utilizes the `DeepSeek-V3-0324` model hosted on Azure OpenAI. The architecture consists of three specialized agents. The Solver, the Verifier and the Planner, all operating within a single LLM context. A key component is the Solver's `PersistentSolverSandbox`, which maintains variable state across turns to allow for iterative code refinement.

Evaluation Protocol. To account for the stochastic nature of LLMs, we adopted a robust evaluation strategy. For each problem in the test set, we performed $K = 4$ independent runs of the full PCAF loop. The reported accuracy for a problem is the average success rate across these runs ($Score \in [0.0, 1.0]$).

Metrics. We assessed performance using three metrics:

- **Accuracy (Pass@K):** The average correctness of the final integer answer across the 4 independent runs.
- **Resilience (Recovery Rate):** The percentage of correct runs where the system initially failed (Turn 1) but successfully self corrected in subsequent turns (Turn 2–4).
- **Baseline Comparison:** Performance is compared against the MathArena's Zero Shot CoT baseline using the same underlying model.

5 Results

As shown in Table 1, the PCAF architecture consistently outperforms the available MathArena baselines. On the AIME 2025 competition our model's framework raised the accuracy from 50.0% to 56.67% On the HMMT Feb 2025 subset, the model's accuracy improved from 29.0% to 45.83%, representing a 16.8% uplift.

Table 1: Performance of PCAF vs. MathArena Baseline (DeepSeek-V3-03-24). **Recovery Rate** denotes the percentage of successful solutions that initially failed on the first turn but were corrected by the agent loop.

Competition Subset	Baseline (0-Shot)	PCAF Accuracy	Recovery Rate	Avg. Turns
AIME 2025	**50.0%**	**56.67%**	29.5%	2.16
BRUMO 2025	N/A	65.83%	48.2%	1.91
CMIMC 2025	N/A	51.25%	41.6%	2.56
HMMT Feb 2025	**29.0%**	**45.83%**	35.0%	2.47
HMMT Nov 2025	N/A	46.67%	25.4%	2.28
SMT 2025	N/A	55.66%	36.7%	2.03
Global Average	N/A	53.65%	**38.9%**	**2.24**

The data highlights a significant "Verification Cost": on average, the system required 2.24 turns to reach a final answer. This latency confirms that the default zero shot reasoning (Turn 1) is frequently insufficient for these high complexity problems, necessitating the intervention of the Verifier and Planner.

5.1 Resilience Analysis

A critical contribution of PCAF is its resilience.

- **One-Shot Success (Turn 1):** 61.1% of correct answers were achieved immediately, typically on problems where the analytical derivation was flawless.
- **Collaborative Recovery (Turns 2-4): 38.9%** of correct answers were "rescued" by the agent loop. Without the PCAF architecture, these instances would have been failures.

This 38.9% recovery rate empirically validates the utility of the Planner's deterministic routing logic.

5.2 Case Study: The "Double Check" Effect

We highlight a representative recovery trace from the HMMT Feb 2025 dataset (Problem ID 1: Sum of divisors of 9! ending in 1) to illustrate the framework's mechanics.

- **Initial Failure (Solver):** The Solver analytically derived divisors $\{1, 81\}$ (Sum: 82), missing the combination $3 \times 7 = 21$.
- **Conflict Detection (Verifier):** The Verifier's dual method check flagged a `METHOD_CONFLICT`: "Text derivation (82) does not match Code Output (103)."
- **Correction (Planner):** The Planner issued a mandatory instruction: "*CRITICAL CONFLICT... Ignore previous text... Re-derive using code evidence.*"
- **Resolution (Turn 2):** The Solver synthesized the code output $\{1, 21, 81\}$, corrected its prime factorization logic, and returned the correct answer of 103.

6 Discussion

The results validate our hypothesis that separating the roles of generation (Solver) and auditing (Verifier) significantly enhances reliability in mathematical reasoning.

Efficacy of the "Constitution of Verification". The recovery rate of 38.9% indicates that the rigid "Constitution" specifically the requirement that code output must match natural language reasoning serves as an effective guardrail. In the HMMT Feb 2025 case study, a standard CoT model would have hallucinated the answer 82 without a mechanism to flag the missing factor. The `PersistentSolverSandbox` acted as an adversarial truth grounding mechanism, forcing the model to confront the reality of its calculation errors.

The Planner's Deterministic Routing. Standard self correction often fails because LLMs are "stubborn" frequently repeating their initial error. The PCAF Planner overcomes this by translating vague Verifier critiques into mandatory, strategy shifting instructions. The high success rate on AIME 2025 Number Theory tasks (where the Planner often forced a "Brute Force" check) suggests that deterministic strategy switching is more effective than open ended "try again" prompting.

Limitations in Geometry. While the system excelled in Algebra and Number Theory (56.7% on AIME 2025), qualitative analysis of failures reveals a weakness in Geometry. When the Solver attempted to parameterize complex shapes (e.g., AIME 2025 Problem 2) in Python, it often struggled to translate visual constraints into correct coordinate code. The Verifier correctly flagged inconsistencies, but the Planner lacked a visual modality to offer spatial correction, leading to loop exhaustion. This suggests that for geometric reasoning, the text based code sandbox must be augmented with Vision Language Models (VLMs).

7 Conclusion

The results of this study validate the Prompt-Based Collaborative Agent Framework (PCAF) as a successful architectural intervention for complex mathematical reasoning. By transitioning from a linear, single-inference process to a modular Solver-Verifier-Planner loop, we have demonstrated that the hallucination and stubbornness inherent in modern Large Language Models (LLMs) are not immutable traits, but can be managed through structured collaboration.

The primary success of the PCAF lies in its ability to enforce Grounding. While standard Chain-of-Thought (CoT) models often succumb to linguistic intuition. The high recovery rate observed in our experiments confirms that the framework's deterministic correction logic effectively rescues solutions that would otherwise fail under standard conditions.

Ultimately, this work proves that for high-stakes reasoning tasks, a model's architecture is as critical as its scale. While the observed limitations in Geometry suggest that a purely text-based sandbox may eventually require visual-modality integration, the current PCAF provides a reliable and transparent blueprint for creating more trustworthy autonomous agents. For researchers and developers aiming to deploy LLMs in technical domains, the PCAF offers a path toward a zero-trust reasoning paradigm where every logic step is critiqued, verified, and, if necessary, strategically corrected.

References

[1] Vincent Li, Yule Fu, Tim Knappe, Kevin Han, and Kevin Zhu. Automating mathematical proof generation using large language model agents and knowledge graphs, 2025. arXiv preprint.

[2] Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal reasoning, 2025. arXiv preprint.

[3] Ruida Wang, Rui Pan, Yuxin Li, Jipeng Zhang, Yizhen Jia, Shizhe Diao, Renjie Pi, Junjie Hu, and Tong Zhang. Ma-lot: Model-collaboration lean-based long chain-of-thought reasoning enhances formal theorem proving, 2025. arXiv preprint.

Appendix A System Prompts

A.1 Solver System Prompt

The Solver is initialized with the following system instruction, which enforces the Dual-Verification Protocol. Note that the prompt explicitly forbids the model from relying on a single method.

> **System:** You are a Rigorous Mathematical Solver. To guarantee accuracy, you must solve the problem using **TWO STRICTLY DIFFERENT METHODS**.
>
> **### PROTOCOL: THE 'ANALYTICAL vs NAIVE' CHECK**
>
> 1. **Method 1: Analytical/Symbolic**: Use `sympy`, algebra, or efficient algorithms to solve the problem mathematically.
> 2. **Method 2: Naive Brute-Force (The 'Dumb' Check)**:
>   - Write a **simple, unoptimized Python loop** that iterates through all possibilities.
>   - **DO NOT** use formulas or clever shortcuts in Method 2. Just count or simulate step-by-step.
>   - *Example*: If counting divisors, Method 1 uses prime factors; Method 2 iterates `for i in range(1, n+1): if n%i==0...`
> 3. **Comparison**: Check if Method 1 == Method 2.
>   - If they MATCH: Print the result.
>   - If they DIFFER: Trust Method 2 (the brute force) or debug Method 1. Do not hallucinate a match.
>
> **### OUTPUT RULES**
>
> - Write all code in ```` ```python ... ``` ```` blocks.
> - You MUST explicitly `print()` the result from BOTH methods.
> - Final Answer must be the value confirmed by BOTH methods.

A.2 Solver Few-Shot Prompts (In-Context Learning)

To teach the Solver how to react to specific Verifier error flags, the system prompt includes three static few-shot examples. These examples explicitly model the "Protocol Reset" behavior required for self-correction.

> **## REFERENCE EXAMPLE: Self-Correction from a VALUE_MISMATCH**
>
> **Problem:** Calculate 5^5 and report the result modulo 100.
>
> **TURN 0: Initial Solve (Flawed)**
> *Solver Logic:* "I calculate 3125. Final Answer is 3125." (Missed the modulo).
> *Verifier Verdict:* `INVALID (VALUE_MISMATCH)`.
> *Planner Instruction:* "The Verifier found mismatch... RE-SOLVE. Apply transformation."
>
> **TURN 1: Solver Correction**
> *Reasoning:* "The Planner noted a VALUE_MISMATCH... The correct final answer must be $3125 \pmod{100}$."
> `print(3125 % 100)` $\rightarrow$ `25`
> *Final Answer:* 25

(Note: The full prompt also includes similar examples for `LOGIC_FLAW` and `METHOD_CONFLICT` to demonstrate how to handle those specific error types.)

A.3 Verifier System Prompt

The Verifier utilizes a structured "Audit Checklist" to classify errors.

> **System:** You are a Skeptical Math Auditor. Your goal is to catch LOGICAL ERRORS that standard code checks miss.
>
> **### AUDIT CHECKLIST**
>
> 1. **The 'Double-Check' Rule**: Did the Solver use **two different approaches** (e.g., Simulation vs. Formula) to confirm the answer?
>   - If they only used one method (e.g., just a formula) and it looks complex/risky, mark as **Risky**.

- If they used two methods and they **CONFLICT**, REJECT immediately.

2. **The 'Consistency' Rule**: Does the [Code Output] match the text? (Crucial).
3. **Logic Trap Checks**:
   - **Combinatorics**: Did they brute force it? If they used a formula like `nCr`, did they verify it for small N?
   - **Geometry**: Did they use coordinates? Visual assumptions are grounds for REJECTION.
   - **Number Theory**: Did they handle edge cases (e.g., 0, 1, negatives)?

OUTPUT FORMAT Return ONLY this JSON: `{"valid": boolean, "error_type": "LOGIC_FLAW" | "VALUE_MISMATCH" | "METHOD_CONFLICT", "critique": string}`

A.4 Planner Instructional Templates

The Planner does not use a single system prompt but explicitly routes error types to specific string templates.

Template 1: Syntax Repair (Triggered by Runtime Errors)
*"The previous code attempt failed with a RUNTIME ERROR... **MANDATORY ACTION:** Fix the syntax. Do not change the math logic yet; just make the code run."*

Template 2: Protocol Reset (Triggered by `VALUE_MISMATCH`)
*"**CRITICAL CONFLICT:** Your text claims one answer, but your Python code calculated a different one. **PROTOCOL:** 1. IGNORE your previous text intuition. It was wrong. 2. **RE-EXECUTE the code** to confirm the calculated value is stable... accept its output as the truth."*

Template 3: General Critique Relay (Triggered by `LOGIC_FLAW` / `METHOD_CONFLICT`)
"The Verifier failed the solution. Reason: {critique}. Please fix the code to match the reasoning."

Appendix B Implementation Details

B.1 The Persistent Sandbox Environment

The `PersistentSolverSandbox` class (defined in `sandbox.py`) provides the runtime environment for the Solver. It differs from standard "code interpreter" tools in two key ways:

- **State Persistence:** The sandbox initializes a dictionary `self.globals` which retains variable state across multiple execution calls. This allows the Solver to define a function in Turn 1 and call it in Turn 2.
- **Safety Wrapper:** Execution is wrapped in the `func_timeout` library with a strict **20-second limit**. This prevents infinite loops (common in naive brute-force attempts) from hanging the evaluation pipeline.

B.2 Input Sanitization

To handle the tokenization artifacts common in mathematical LLM outputs, the sandbox implements a pre-execution sanitization layer:

```
code_str = code_str.replace("≈", "=")
code_str = code_str.replace("≠", "!=")
code_str = code_str.replace("≤", "<=")
code_str = code_str.replace("≥", ">=")
```

Appendix C Recovery Trace Case Study

The following trace illustrates a successful self-correction from the HMMT Feb 2025 dataset (Problem 12). It demonstrates how the PCAF architecture breaks "Reasoning Inertia"—the tendency of LLMs to reject correct code outputs that contradict their internal intuition.

Problem: Kevin the frog in on the bottom-left lily pad of a 3×3 grid of lily pads, and his home is at the topright lily pad. He can only jump between two lily pads which are horizontally or vertically adjacent. Compute the number of ways to remove 4 of the lily pads so that the bottom-left and top-right lily pads both remain, but Kelvin cannot get home.

TURN 0: Initial Attempt (Failure)

- **Solver Strategy:** Tries to solve using both Brute-Force and Analytical methods.
- **Initial Code Output:** The Brute-Force script correctly calculates **29**.
- **Solver Hallucination:** The Solver rejects the code's result because it contradicts its flawed analytical intuition (which guessed 5).
- **Solver Text Reasoning:** "The first run of the corrected brute-force method (29) is incorrect... The second run (5) matches the analytical method... confirming the correct answer is 5."
- **Final Answer (Turn 0):** 5.

VERIFICATION (The Intervention)

- **Verifier Verdict:** `INVALID`
- **Error Type:** `METHOD_CONFLICT`
- **Critique:** "The solver initially reported conflicting results between Method 2 (Brute-Force: 29) and Method 1 (Analytical: 5)... The intermediate output of 29 indicates a lack of reliable dual-verification. The final agreement is only achieved after corrections, suggesting the initial logical approach was flawed."

PLANNING (The Instruction)

- **Planner Action:** Triggered **Critique Relay**.
- **Instruction:** "The Verifier failed the solution... The initial inconsistency and intermediate output of 29 indicate a lack of reliable dual-verification... Please fix the code to match the reasoning."

TURN 1: Correction (Success)

- **Solver Reasoning:** "I see the persistent discrepancy... Method 2 (Brute-Force) enumerates all possibilities and correctly counts **29** disconnecting configurations. My previous analytical method underestimated the count."
- **Solver Conclusion:** "Both methods now consistently agree on the correct count of 29."
- **Final Answer:** 29.

Appendix D Failure Case Study: Geometry (The "Blind Spot")

While PCAF excels at Algebra, this case study from the AIME 2025 dataset (Problem 2) illustrates a structural limitation. In Run 2, the Solver hallucinated a spatial constraint that the text-based Verifier could only catch via logical deduction, not visual inspection.

Problem (AIME I 2025, Problem 2): Points M and N lie on sides AB and AC of $\triangle ABC$. A heptagon is formed by removing specific triangles. Given the area of $\triangle ABC$ is 588, determine the area of the heptagon.

TURN 0: Initial Attempt (Failure)

- **Solver Strategy:** Coordinate Geometry (Shoelace Formula).
- **Code Output:** The code calculated an area of **588**.

- **Solver Hallucination:** The Solver effectively guessed the answer was the same as the total area, justifying it with a spatial impossibility.
- **Solver Text Reasoning:** "Looking at the problem again, it's clear that the heptagon AFNBCEM is the entire triangle ABC. Therefore, the area is indeed 588."
- **Final Answer (Turn 0):** 588.

VERIFICATION (The Critique)

- **Verifier Verdict:** `INVALID`
- **Error Type:** `LOGIC_FLAW`
- **Critique:** "The solver's reasoning is flawed because they incorrectly concluded that the heptagon AFNBCEM has the same area as the entire triangle ABC. The heptagon should be a part of the triangle, not the whole triangle. The solver's repeated use of the shoelace formula without correctly identifying the vertices led to a logical error."

OUTCOME (Loop Exhaustion)

- **Failure to Recover:** Because the Planner lacks a Vision-Language Model (VLM), it could not "show" the Solver why the heptagon was a subset. The Solver spent the next 3 turns randomly perturbing coordinates before failing.
- **Note:** This is a "Right Answer, Wrong Reason" failure. The ground truth *was* 588, but the Solver's claim that "Heptagon = Triangle" was semantically false (the Triangle area was likely larger in the problem's implicit geometry), causing the Verifier to rightly reject the reasoning.